RAZOR: Learning What to Forget in the KV Cache with a Description-Length Price

Himansu Rawal
Manaslu Labs

Abstract

The key–value (KV) cache of a transformer grows linearly with context length, and at long context it takes more GPU memory than the model weights. We present RAZOR, a method for training a small gate that decides, when each token is written to the cache, whether to keep it. The base model stays frozen. The gate is trained to leave the model's output distribution unchanged, measured as KL divergence to the uncompressed model, while paying a price for every token it keeps. The price is updated by dual ascent until a target memory budget is met, so no penalty weight has to be tuned. Keep/drop decisions are binary during training, with straight-through gradients, and the gate therefore trains on the same cache it produces at inference. Trained for 39 minutes on one T4 GPU with documents of at most 1k tokens, the gate (1.7% of model parameters) lets Qwen3-1.7B evict 76% of its KV cache on RULER-4k while scoring 86.6 ± 0.6 over five seeds, against 88.9 without compression and 86.5 for KVzip+, which requires a second pass over the context. With architecture and data held fixed, it outperforms a gate trained on KVzap's regression targets by 2.3–7.1 points. The same gate transfers to 16k-token RULER and 100k-token InfiniteBench, and lets Qwen3-4B read 123k tokens on a 16 GB GPU, where the uncompressed model runs out of memory at 61k; at 29k tokens, decoding is 2.9× faster. Finally, across four budgets the learned price matches the slope of the cache's rate–distortion curve, which gives a direct measure of what a unit of KV memory is worth to the model.

1 Introduction

Long-context inference is memory-bound. For Qwen3-4B in fp16, 128k tokens of KV cache take ∼19 GB, more than the 8 GB of weights and more than an entire T4 GPU. Most KV compression methods decide what to drop with a hand-designed proxy: recent tokens [29], accumulated attention [32], attention from an observation window [20], predicted future attention [6], or how well the context can be reconstructed from the compressed cache [17]. The last is the strongest but costs a second pass over the context.

A fast-moving line of work replaces the proxy with a learned gate that scores each token when it is written [3, 10, 13, 14, 16, 18, 19, 33]. Two families have emerged. *Distilled* gates regress a hand-designed target: KVzap [16] and Fast KVzip [18] fit KVzip's reconstruction scores, so their quality is capped by the proxy they imitate. *End-to-end* gates train against the frozen model's own outputs [3, 10, 13, 33]; to make the keep/drop decision differentiable they relax it, into an exponential decay inside the softmax (TrimKV, IndexMem++), a continuous admission mask (WG-KV) or an annealed Gumbel top-k (LKV), and they control memory with a penalty weight that is swept, a fixed retained mass, or an exact top-k. Their reported setups train on 4k–128k-token documents with datacenter GPUs.

In this work we train the gate without any relaxation. It makes the same hard 0/1 decisions in training as at inference, and memory is controlled by a dual-ascent price, a standard tool in constrained sparsity [8, 28], in place of a swept penalty weight. The objective can be read as a two-part description length [9, 26]: kept tokens cost memory, dropped tokens cost excess code length on the future, measured as KL divergence to the uncompressed model, and RAZOR minimizes the sum.

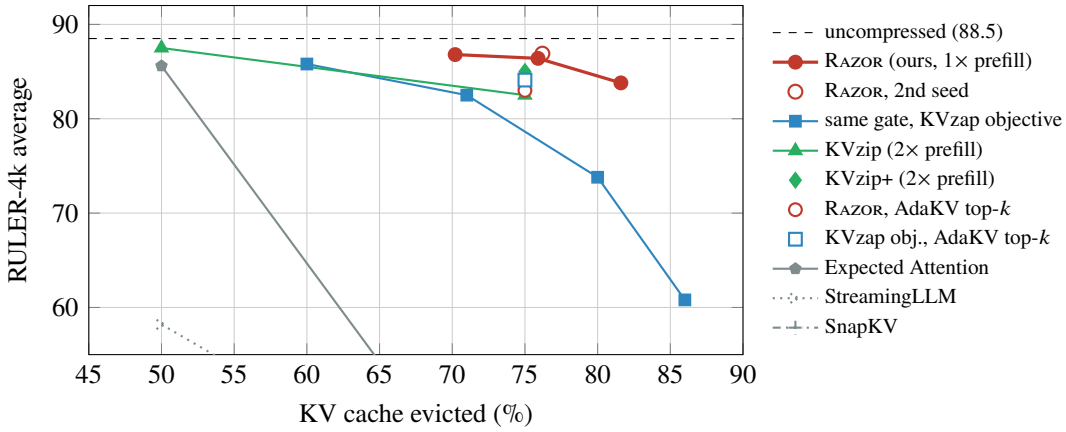


Figure 1: **Quality vs. memory on RULER-4k** (Qwen3-1.7B, n=195, query-agnostic compression; n=650 confirmation in §3). Lines are threshold sweeps or fixed ratios; open markers use per-head adaptive top-k (AdaKV-style). Razor's threshold sweep stays within 2.2 points of the uncompressed model up to 76% eviction. The same gate trained to regress KVzap's targets falls to 73.8 at 80% under thresholding, but reaches 84.1 with AdaKV top-k at 75%. At 75%, Expected Attention (33.3), StreamingLLM (35.9) and SnapKV (18.8) fall below the plotted range; these query-aware methods are disadvantaged by the query-agnostic protocol.

Contributions.

- **A relaxation-free recipe.** Exact hard masks with straight-through gradients [2], KL to the frozen model, and a dual-ascent price that hits a target keep rate without sweeping a penalty weight. With binary decisions and an enforced budget, a constant gate cannot be a no-op, the failure mode that motivates in-softmax relaxations [10]. The loss reads as a two-part code: excess nats on the future plus the size of the stored memory (§2).
- **The price is measurable.** Across gates trained at four budgets, the converged dual price lies within the convexity bracket of the slope of the KV cache's rate–distortion curve at every budget, rising 68-fold from 50% to 10% kept. Retraining for a budget beats moving a gate's threshold by up to 3.5× in KL (§6).
- **Controlled comparisons.** With architecture, data and hyperparameters fixed, training on output fidelity beats regressing KVzap's targets by 7.1 points at 4k and 11.8 at 16k under threshold inference, and is stable across five seeds (86.6 ± 0.6); with each gate in its best inference mode the gap is 2.3 points. With the gate fixed, thresholding at the training rule beats uniform fixed-ratio top-k by 24 points and AdaKV-style top-k by 3.4 (§3, §7).
- **Length transfer at tiny cost.** Gates trained for 39 minutes on ≤1k-token documents remain usable at 16k (RULER) and 100k tokens (InfiniteBench, n=12 per task) (§3, §4).
- **Long context on a consumer GPU.** With per-head ragged deletion and chunked prefill, Qwen3-4B reads 123k tokens on a single 16 GB T4, extending the consumer-GPU setting of Locret [14] to a frozen model with a write-time gate (§4).
- **Negative results.** Five variants that did not help, reported in full (§7).

2 Method

Gate. For each layer ℓ, a two-layer MLP $g_\ell : \mathbb{R}^d \to \mathbb{R}^{H_{kv}}$ (hidden width 512) reads the input hidden state $u_t^{(\ell)}$ of the attention block and outputs a logit $s_{t,h}^{(\ell)}$ for every KV head h. This is exactly the KVzap architecture [16], so any difference we measure comes from the training signal. For Qwen3-1.7B (28 layers,

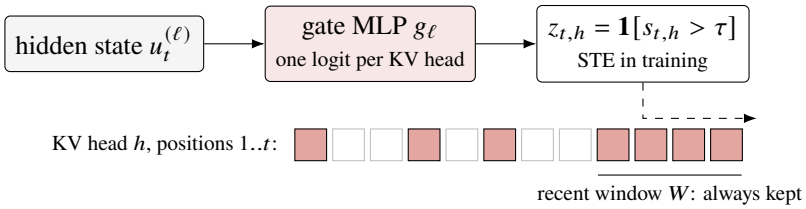


Figure 2: **Razor at inference.** When token t is written at layer ℓ, a small MLP reads its hidden state and emits one logit per KV head. Tokens whose logit is below τ are deleted from that head's cache once they leave the recent window of W=128 tokens (white cells). Each head keeps a different, content-dependent set of tokens.

8 KV heads) the gate has 29M parameters (1.7% of the model); for Qwen3-4B, 47M. The base model is frozen.

Hard, exact eviction during training. Let $p_{t,h} = \sigma(s_{t,h})$ and $z_{t,h} = \mathbf{1}[p_{t,h} > \frac{1}{2}]$. In the forward pass, attention from query i to key j in head h is

$$a_{ij}^{(h)} = \frac{e^{q_i^\top k_j/\sqrt{d_k}}\, m_{ij}^{(h)}}{\sum_{j'\le i} e^{q_i^\top k_{j'}/\sqrt{d_k}}\, m_{ij'}^{(h)}}, \qquad m_{ij}^{(h)} = \begin{cases} 1 & i-j < W \\ z_{j,h} & i-j \ge W \end{cases} \tag{1}$$

computed in fp32. With hard z this equals deleting key j from head h's cache once it leaves the window, which is exactly the decode-time inference rule. (During prefill at inference, keys are evicted only after their layer is prefilled, so hidden states of the context itself are computed with full attention; training computes them under the mask. The decisions match; their inputs differ slightly.) In the backward pass we use the straight-through estimator $\partial z/\partial p := 1$ [2].

Objective: an MDL-inspired two-part loss. Let P be the next-token distribution of the uncompressed model and Q_z that of the model running on the gated cache. Over the last N positions $i \in \{T-N+1, \dots, T\}$ of a training document of length T,

$$\mathcal{L}(g) = \underbrace{\frac{1}{N}\sum_i \mathrm{KL}(P(\cdot \mid x_{<i}) \,\|\, Q_z(\cdot \mid x_{<i}))}_{\text{excess code length of the future}} + \lambda \underbrace{\frac{1}{LHT}\sum_{\ell,h,t} p_{t,h}^{(\ell)}}_{\text{length of the memory}}. \tag{2}$$

The first term is the extra nats per token needed to encode the continuation when the model sees the compressed cache instead of the full one. The second is the expected fraction of tokens kept, over all layers, heads and positions; it is proportional to the bytes the cache occupies, so λ converts memory into nats and acts as a price per unit of memory. The penalty uses the soft p so that it has a gradient; the constraint below is on the hard decisions actually executed. Rather than tuning it, we treat the budget ρ as a constraint and update the price by projected dual ascent after every step:

$$\lambda \leftarrow \max\big(0,\ \lambda + \eta\,(\bar{z} - \rho)\big), \tag{3}$$

where $\bar{z}$ is the *hard* keep rate over all gated positions of the current document. This is the standard Lagrangian approach to controlled sparsity [8]. Memory gets more expensive while the gate keeps too much and cheaper once it is under budget. We use ρ=0.3, η=0.5.

Training setup. 1,282 documents from the same Nemotron mixture used to train KVzap (5 subsets), 750–1,250 tokens each, truncated to 1,024; KL over the last N=512 positions; window W=128; one epoch at batch size 1 (1,282 steps); AdamW, lr 10^{-3}, cosine schedule, gradient clipping 1.0; output bias initialized to 1.5 so the gate starts mostly open ($p \approx 0.82$). Gradient checkpointing lets the full fp32-attention graph fit in 15 GB. Training takes **39 min** (1.7B) and **1.7 h** (4B) on one T4. The final keep rate is 32% at KL $\approx$ 0.01 nats/token.

2.1 Inference: keep what the gate keeps

At inference a token is kept in head h iff $s_{t,h} > \tau$ (DMS-style thresholding [19]) or it lies in the recent window. τ=0 is exactly the training rule $p > \frac{1}{2}$. How many tokens each head keeps therefore depends on the content, and the global eviction rate is an output, not an input. Sweeping $\tau \in \{-1, 0, 1\}$ traces the curve in Figure 1. The training budget ρ does not fix the deployed keep rate: at τ=0 the gate keeps 24% at 4k and 16–24% at 16k–100k, so a different operating point is chosen by τ at inference. The RULER and MATH results use kvpress's masking with these decisions; the memory and InfiniteBench results use the ragged cache below.

Ragged cache. Masking evicted keys, as most KV compression libraries do for evaluation, saves no memory. We store each KV head's surviving keys, values and positions in its own preallocated buffer and run attention per head over its ragged cache, which is numerically identical to masking (max abs. logit difference 1.5×10^{-7}). Eviction happens layer by layer as soon as that layer is prefilled, so peak memory is the kept cache plus one uncompressed layer. With *chunked prefill* (4,096-token chunks), each chunk attends only to what earlier chunks kept, which bounds activation memory as well. With nothing evicted, chunked prefill reproduces one-shot prefill to 10^{-7}.

3 Main results

Setup. Models: Qwen3-1.7B and Qwen3-4B [25], fp16. Benchmarks: RULER [12] at 4k and 16k (13 tasks), InfiniteBench [31] truncated to 100k tokens, MATH-500 [11, 21]. All compression baselines run through NVIDIA kvpress [23] with identical prompts, data subsets and scoring. Every experiment runs on free 2×T4 Kaggle instances.

Protocol. Compression is query-agnostic: the context is compressed during prefill and the question is appended afterwards. Eviction rates are the fraction of all cached KV entries removed, counting the recent window as kept.

The key control. NVIDIA has not released a KVzap gate for Qwen3-1.7B, so we train our own: the *same* architecture, data and hyperparameters, with KVzap's objective (regress KVzip+ scores, KVzap's improved variant of KVzip's reconstruction scores). Its threshold (−4.5) was chosen on the n=195 sweep to match Razor's eviction rate. We refer to it as "same gate, KVzap objective"; it isolates the effect of the training signal.

RULER-4k. At 76% eviction (Table 1), Razor loses 2.2 points to the uncompressed model and ties KVzip+, which re-reads the context to score it. On needle retrieval it is near-lossless (97.9 vs. 98.5) and beats KVzip+ by 3.4. Holding architecture and data fixed, changing only the objective from "imitate KVzip+" to "preserve the output distribution" gains 7.1 points under threshold inference. The gap is smaller when each gate uses its best mode: with AdaKV-style top-k the KVzap-objective gate reaches 84.1 at 75% (n=195), 2.3 below Razor's threshold mode (86.4). Part of Razor's advantage is that its scores are calibrated decisions that can be thresholded directly. Razor's weakness is common-word extraction (CWE), an aggregation task

Table 1: **RULER at matched eviction** (Qwen3-1.7B). RAZOR and the KVzap-objective gate run in threshold mode; KVzip+ at a fixed ratio (kvpress default allocation). "Prefill" is passes over the context; KVzip+'s evaluation took 2× RAZOR's wall time. NIAH = mean of the 8 needle tasks.

	Method	Evicted	Prefill	**Avg**	NIAH	VT	CWE	FWE	QA-1	QA-2
	Uncompressed	0%	1×	88.9	98.5	99.0	74.7	80.7	72.3	40.9
4k	KVzip+	76%	2×	86.5	94.5	98.0	**79.1**	**76.0**	**76.6**	**38.6**
n=650	Same gate, KVzap obj.	76%	1×	79.6	89.1	74.2	71.6	74.0	68.1	34.1
	RAZOR	76%	1×	**86.7**	**97.9**	**99.0**	57.4	74.7	**76.6**	36.4
	Uncompressed	0%	1×	77.9	89.2	100	12.5	85.7	73.3	27.8
16k	KVzip+	76%	2×	73.6	85.3	93.3	**22.5**	**88.1**	53.3	16.7
n=195	Same gate, KVzap obj.	78%	1×	63.6	76.5	60.0	8.3	76.2	53.3	16.7
	RAZOR	81%	1×	**75.4**	**88.1**	**95.6**	5.8	85.7	**60.0**	**27.8**
4k, 4B	Uncompressed	0%	1×	94.3	100	100	95.0	83.3	86.7	61.1
n=195	RAZOR	74%	1×	93.9	100	100	88.3	78.6	86.7	66.7

where every occurrence of a frequent word matters. A gate trained to preserve next-token predictions on ordinary text has little reason to keep the 30th repetition of a word.

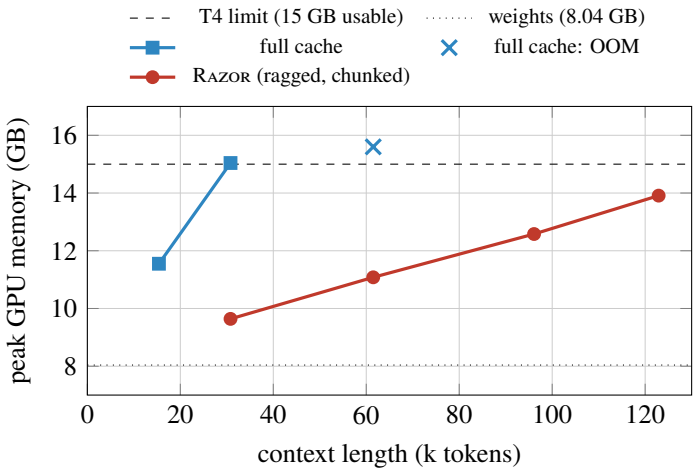
Length generalization to 16k. The gates never saw a document longer than 1,024 tokens. At 16k, RAZOR evicts *more* than any baseline (81%) and is statistically tied with KVzip+ (75.4 vs. 73.6, n=195) with one prefill instead of two, 2.5 below uncompressed. The KVzap-objective gate falls 14.3 points, and its gap to RAZOR widens from 7.1 at 4k to 11.8. Retention decisions learned from local hidden states appear to be largely length-independent.

Scale: Qwen3-4B. The 4B gate, trained with the identical recipe, scores 93.9 vs. 94.3 uncompressed on RULER-4k at 74% eviction (Table 1, n=195; no compressed baselines were run at 4B). All 8 needle tasks and VT stay at 100. The losses are again CWE (95→88) and FWE (83→79), with QA-2 up 5.6.

Robustness across seeds. Five gates trained from different seeds give 86.6 ± 0.6 on RULER-4k (n=650 each; 86.7, 87.0, 86.4, 85.8, 87.4) at 76.3–76.7% eviction; three of the five are above KVzip+ (86.5). Under thresholding, the recipe is stable: the seed-to-seed spread is smaller than the gap to any baseline except KVzip+.

4 Long context on one GPU

Memory. Figure 3 shows what physical deletion buys. On one T4, uncompressed Qwen3-4B fits ~31k tokens with one-shot prefill (chunked prefill would stretch this to an estimated ~40k, since its 144 KiB/token cache still has to fit). With RAZOR it reads 123k tokens (the limit of the 4× YaRN [24] extension) and returns the correct passkey at every length. At 61k the cache is 1.41 GB instead of ~9 GB. Prefill of 123k tokens takes 222 s on a T4. Decoding is faster too, because each new token attends to a cache a sixth of the size. In a separate run (single sample per length, eager attention on a T4), uncompressed decode slows from 8.8 to 2.6 tokens/s between 8k and 29k tokens, while RAZOR holds 8.2–7.5 tokens/s over the same range (2.9× faster at 29k, with prefill also 2.0× faster: 43 s vs. 85 s) and still decodes 4.5 tokens/s at 61k and 2.7 at 115k, lengths the uncompressed model cannot reach. At 8k, where the cache is small, RAZOR is 7% slower than uncompressed because of the gate and the ragged-cache bookkeeping.



tokens	KV (GB)	passkey
31k	0.72	✓
61k	1.41	✓
96k	2.22	✓
123k	2.84	✓

RAZOR on Qwen3-4B, 84% evicted. Uncompressed KV at 123k would be ~18 GB. One passkey per length.

Figure 3: **Longest context on a single 16 GB T4** (Qwen3-4B, fp16, YaRN×4, passkey in a filler haystack). The uncompressed model (one-shot prefill) peaks at 15.04 GB at 31k and fails at 61k; the OOM marker's height is illustrative. RAZOR reads 123k tokens with a 2.8 GB cache and peak memory still under the limit. Peak is now dominated by weights and prefill activations, not the cache.

Table 2: **InfiniteBench, 100k tokens** (Qwen3-4B, YaRN×4, chunked prefill, n=12 per task). The uncompressed reference runs over two T4s; RAZOR runs on one.

Task	Uncompressed (2 GPUs)	RAZOR (1 GPU)	Evicted
passkey	0.67	0.67	84%
number_string	0.67	0.67	84%
longbook_choice	0.58	0.67	76%
kv_retrieval	0.08	0.08	78%

InfiniteBench at 100k. The gates were trained on ≤1k-token documents and the model was extended to 100k only through YaRN. Table 2 compares RAZOR on a single T4 to the uncompressed reference, which must be split across two. With n=12 per task, RAZOR is within one item of it on every task. On passkey and number_string it answers exactly the same items correctly; on longbook_choice it gets one more item right. kv_retrieval is at floor for both: Qwen3-4B cannot do it at 100k even without compression.

5 Forgetting while thinking

Reasoning models make the decode-time cache, not the prompt, the bottleneck: a single MATH answer from Qwen3-1.7B averages ~5k generated tokens. We apply RAZOR *during generation*: every new token is gated as it is written. We use 30 MATH-500 level 4–5 problems, sampling at T=0.6, top-p 0.95, an 8k-token cap, and identical seeds for all methods.

The general gate, trained only on short general-domain documents, evicts three quarters of the cache during reasoning and never loses a problem the uncompressed model solves (Table 3). With n=30 and one sample per problem the +6.7 point difference is not significant, so we read this as *no loss*, not as a gain. A gate trained on DeepSeek-R1 traces [5, 15] showed no benefit from in-domain training at this sample size. We attempted a head-to-head with TrimKV's released math gate on this setup, but its official implementation depends on FlashAttention, which T4 GPUs lack; our SDPA port produced degenerate output even with eviction disabled, so we report no TrimKV numbers.

Table 3: **Compressing the cache while the model reasons** (Qwen3-1.7B, MATH-500 L4–5, n=30, 8k cap). "Paired" compares per problem with the uncompressed run: both correct / only this method / only uncompressed.

Method	Evicted	Accuracy	Paired	Avg. tokens
Uncompressed	0%	66.7%	–	5,029
RAZOR (general gate)	75.4%	73.3%	20 / 2 / 0	5,218
RAZOR (gate trained on R1 math traces)	70.1%	70.0%	18 / 3 / 2	5,265

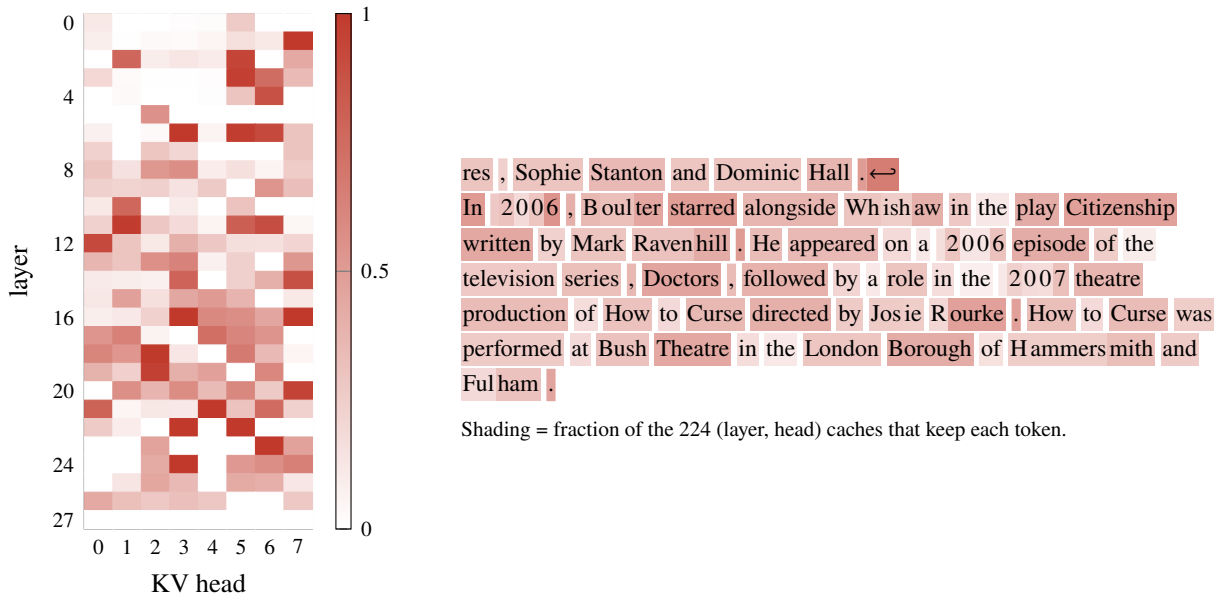


Figure 4: **What the Qwen3-1.7B gate keeps** (16 wikitext documents of 2,048 tokens, τ=0, tokens outside the recent window). *Left:* keep rate per layer and KV head. Heads specialize: 18 of 224 keep over 90% of tokens and 73 keep under 5%; the last layer keeps nothing beyond its window. *Right:* a passage, shaded by how many caches keep each token. Line breaks and content words survive; articles mostly do not.

6 What does the gate learn?

We ran the 1.7B gate over 16 wikitext-103 documents of 2,048 tokens and recorded every keep/drop decision outside the recent window (Figure 4). Overall it keeps 28% of tokens, close to the 32% it was trained to keep.

Heads become retrieval or streaming heads. The per-head keep rates are strongly bimodal: 18 heads (8%) keep more than 90% of what they see and 73 (33%) keep less than 5%, effectively becoming sliding-window heads. The 18 near-full heads hold 28% of all retained KV. This is the split between retrieval and streaming heads that DuoAttention [30] identifies and trains for explicitly; here it emerges from a per-token objective with no head-level supervision. The final layer keeps nothing outside its window, and layers 0 and 5 keep under 10%.

Structure over filler. Averaged over all caches, line breaks are kept most (57%), followed by capitalized words (34%), other content words and punctuation (32%). Articles, prepositions and other function words are kept least (18%), as are whitespace tokens (17%). Number tokens are kept less than average (24%): Qwen3 splits numbers into single digits, and in the example passage the gate keeps the last digit of a year more often

Table 4: Rate–distortion check of the dual price. One gate per budget ρ; keep rate $\bar{z}$, training KL (nats/token) and λ are means over the last 500 training steps. For a convex curve the slope at a point lies between the secants to its neighbours; λ lands in that bracket at all four budgets.

ρ	keep $\bar{z}$	KL	price λ	secant bracket on $-d\mathrm{KL}/d\bar{z}$
0.1	0.101	0.048	0.58	≥ 0.30
0.2	0.198	0.020	0.11	[0.069, 0.30]
0.3	0.298	0.013	0.047	[0.038, 0.069]
0.5	0.487	0.005	0.0085	≤ 0.038

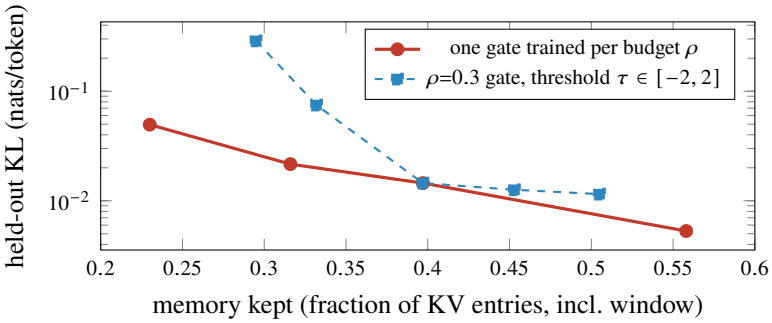


Figure 5: Rate–distortion curve of the KV cache for Qwen3-1.7B on 45 held-out documents (1k tokens). Training a gate for the budget dominates moving one gate's threshold, by up to 3.5× in KL when evicting more.

than the middle ones (Figure 4, right). That is a plausible reason for RAZOR's weakness on aggregation tasks such as CWE, which require every occurrence of a token.

The price is the slope of the rate–distortion curve. If dual ascent converges, λ is the Lagrange multiplier of the keep-rate constraint, and the optimality condition says it should equal the marginal distortion of memory, $-d\mathrm{KL}/d\bar{z}$, at the trained budget. We test this by training gates at $\rho \in \{0.1, 0.2, 0.3, 0.5\}$ with everything else fixed and tracing the curve they span (Table 4). The curve is convex, so its slope at each point is bracketed by the secants to its neighbours. The converged price falls inside that bracket at every budget, and it rises 68-fold, from 0.0085 to 0.58 nats per unit of memory, as the budget tightens from 50% to 10%. The multiplier therefore measures the exchange rate between memory and code length, and indicates how much a further reduction in budget will cost before a gate is trained for it.

The curve also generalizes and separates training from thresholding (Figure 5). On 45 held-out documents, the KL of each gate matches its training KL to within 0.003 nats. Moving one gate along its own curve by shifting the threshold τ is far worse than training a gate for the new budget when evicting more: the ρ=0.3 gate at τ=1 keeps 33% of memory at 0.075 nats, while the ρ=0.2 gate keeps 32% at 0.022, 3.5× less distortion. When keeping more, thresholding is nearly free (τ=−1 lands on the retrained curve). A deployment that needs a tighter budget should retrain, which takes 39 minutes.

7 What did not work

We report variants we expected to help and that did not. All are on RULER-4k, Qwen3-1.7B, n=195, evaluated with uniform fixed-ratio top-k at 75%, single seed. Because this evaluation mode varies by 28 points between two seeds of the same recipe, only the large gaps are conclusive.

Table 5: **Variants that did not help** (RULER-4k, Qwen3-1.7B, n=195, uniform top-k at 75% unless noted).

Variant	RULER-4k @75%	What happened
RAZOR, fixed-ratio inference (seed 0)	62.0	reference for this table
same, seed 1	33.8	fixed-ratio ranking is seed-fragile
no MDL price (λ=0)	0.4	gate keeps 99%; forced ranking is meaningless
soft gates (no STE)	38.1	within seed range; inconclusive
trained on 2k-token documents	15.8	only 499 docs; undertrained
KL on hardest 5% of positions	54.5	within seed range; inconclusive
context-reconstruction objective	11.8	KL dominated by trivially copied tokens
RAZOR, AdaKV-style top-k	83.0	per-head adaptive budget
RAZOR, threshold inference (τ=0)	**86.4** (76% evicted)	seed 1: 86.9

A gate trained to threshold should be evaluated by thresholding. Most evaluation harnesses force every method to evict exactly r% of tokens by taking a top-k over scores. RAZOR is never trained to rank tokens, only to decide $p > \frac{1}{2}$. Under a forced ratio its quality depends on the ordering of scores far from the decision boundary, which the objective does not constrain. That ordering varies from seed to seed (62.0 vs. 33.8). Under thresholding the same two gates are within 0.5 points. With per-head adaptive budgets at a fixed 75% (AdaKV-style [7]), RAZOR (83.0), the KVzap-objective gate (84.1) and KVzip+ (85.0) are close. Much of the advantage comes from letting the gate decide *how much* to keep, per head and per input. One caveat: under thresholding the eviction rate is a mean over inputs (71–85% per input, 5th–95th percentile), whereas fixed-ratio baselines get the same budget on every input.

Hard vs. soft gates. We trained the same gate with sigmoid gates in the forward pass (no straight-through) and swept its threshold to compare at matched eviction on the same 195 RULER-4k samples. At 75.8% evicted the soft gate scores 84.8 against the hard gate's 86.4 at 75.9%; at 79% it scores 82.6, about 2.3 below the hard gate's interpolated curve; at 72% the two tie (86.6 vs. ≈86.7). The direction favours hard gates, increasingly so as eviction grows, but the gaps are within the ∼3-point noise of n=195. We read this as: training on the exact decisions used at inference is at least as good as a relaxation, and simpler, not as a proven win.

Without the price, the gate keeps everything. Without it the KL term alone is minimized by keeping everything: the gate learns to keep 99% of tokens and the method collapses when forced to evict.

Query-agnostic reconstruction is a poor teacher. Asking the compressed model to restate its context (KVzip's criterion, used here as a training loss) yields a tiny mean KL (0.002), since most tokens are easy to copy, and gives the gate almost no signal about which tokens matter. Restricting the KL to the hardest positions did not rescue it.

8 Related work

Heuristic eviction. StreamingLLM [29] keeps attention sinks and a recent window; H_2O [32] and SnapKV [20] keep tokens with high observed attention; Expected Attention [6] estimates attention from future queries; Ada-KV [7] allocates budgets per head, and DuoAttention [30] learns which heads need the full context. KVzip [17] scores tokens by how much the model attends to them while reconstructing the context, at the cost of a second pass.

Learned retention. Dynamic Context Pruning [1] learns to drop tokens end to end by fine-tuning the model. Locret [14] trains cheap retaining heads and runs 128k contexts on a single consumer GPU, and DMS [19] retrofits delayed eviction by fine-tuning the model itself. KVzap [16] and Fast KVzip [18] distill KVzip scores into light gates; Fast KVzip reports that reconstruction targets generalize better than next-token targets, whereas in our end-to-end setting a reconstruction objective was the worst variant we tried (§7). Closest to RAZOR are gates trained against a frozen model's outputs. TrimKV [3] uses a hidden-state MLP whose retention score acts as an exponential decay inside the softmax, trained with KL plus next-token loss and a hinge capacity penalty, and evicts to a fixed per-head budget. WG-KV [13] learns a write-time admission gate on keys with a continuous in-softmax mask, hidden-state MSE distillation and a swept sparsity weight, then thresholds it into a ragged paged cache. LKV [33] learns per-head budgets and token scores with an annealed Gumbel Soft-TopK under KL plus hidden-state distillation. IndexMem++ [10], concurrent with this work, uses the same hidden-state MLP scorer and window as RAZOR with reverse-KL distillation, and argues for placing a soft gate inside the softmax with a fixed retained mass instead of straight-through estimation. RAZOR differs in using exact hard masks with a dual-ascent keep-rate constraint instead of a relaxation, in thresholding at the training rule, and in training on $\leq$1k-token documents on a single consumer GPU. A head-to-head under a common protocol is future work; the reported numbers of these methods use different models, lengths and budget accounting and are not directly comparable to ours.

Orthogonal directions. Quantization (e.g. KIVI [22]), query-aware sparse retrieval (Quest [27]) and architectural changes such as latent attention [4] reduce bytes per token or tokens read per step. RAZOR reduces tokens stored and composes with all of them.

9 Limitations

- **Scale.** We evaluate 1.7B and 4B models on free T4 GPUs; behaviour at larger scale is untested.
- **Sample sizes.** Figure 1, §7 and RULER-16k use n=195 in total (15 per task); InfiniteBench uses n=12 per task; MATH uses n=30 with one sample each. Differences under ~3 points at n=195 are within noise.
- **Aggregation tasks.** CWE degrades at every length. A gate that cannot know the future query will drop redundant-looking repetitions.
- **Baselines.** Our KVzap comparison is a re-training with RAZOR's data and hyperparameters, not NVIDIA's checkpoint (none exists for Qwen3-1.7B) and not KVzap's own training recipe. No compressed baselines were run at 4B. We do not yet compare against the end-to-end gates of WG-KV, LKV or IndexMem++ under a common protocol, and could not run TrimKV's released code on our hardware (§5).
- **Hard vs. soft.** At matched eviction hard gates lead soft ones by up to 2.3 points, within the noise of n=195; a larger sample is needed to call this a win.
- **Budget.** Thresholding gives a mean eviction rate, not a per-input guarantee: on RULER-4k, per-input eviction ranges from 71% to 85% (5th–95th percentile) around the 76% mean, so memory planning must provision for the low end.
- **Kernels.** The ragged attention is a reference PyTorch implementation at batch size 1. A fused kernel and paged-memory integration (vLLM/SGLang) are future work.

10 Conclusion

We presented RAZOR, a write-time gate for KV cache eviction trained against a frozen model with KL divergence and a dual-ascent price on memory. A 29M-parameter gate, trained for 39 minutes on one T4, evicts three quarters of the cache, matches KVzip+ with a single prefill, transfers from 1k-token training documents to 100k-token inputs, and lets a 4B model read 123k tokens on a 16 GB GPU. The learned price tracks the

slope of the cache's rate–distortion curve. A natural extension is to price precision as well as presence, so that the gate chooses how many bits each kept token is stored with.

References

[1] Sotiris Anagnostidis, Dario Pavllo, Luca Biggio, Lorenzo Noci, Aurelien Lucchi, and Thomas Hofmann. Dynamic context pruning for efficient and interpretable autoregressive transformers. In *Advances in Neural Information Processing Systems*, 2023.

[2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[3] Ngoc Bui, Shubham Sharma, Simran Lamba, Saumitra Mishra, and Rex Ying. Cache what lasts: Token retention for memory-bounded KV cache in LLMs. In *International Conference on Learning Representations*, 2026. arXiv:2512.03324.

[4] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[5] DeepSeek-AI. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025. arXiv:2501.12948.

[6] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025.

[7] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2407.11550.

[8] Jose Gallego-Posada, Juan Ramirez, Akram Erraqabi, Yoshua Bengio, and Simon Lacoste-Julien. Controlled sparsity via constrained optimization or: How I learned to stop tuning penalties and love constraints. In *Advances in Neural Information Processing Systems*, 2022.

[9] Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[10] Hao Gu, Xintong Yang, Zhiwei Li, Lei Zhu, Jian Yang, Qiyuan Zhu, Dezhi Li, Jingyuan Zhang, Zhizhuo Kou, Hao Wang, Bei Liu, Jiacheng Liu, and Sirui Han. The gating mechanism of KV cache eviction. *arXiv preprint arXiv:2611.07605*, 2026.

[11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021.

[12] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Conference on Language Modeling*, 2024.

[13] Yen-Chieh Huang, Pi-Cheng Hsiu, Rui Fang, and Ming-Syan Chen. KV admission: Learning what to write for efficient long-context LLM inference. *arXiv preprint arXiv:2512.17452*, 2025.

[14] Yuxiang Huang, Binhang Yuan, Xu Han, Chaojun Xiao, and Zhiyuan Liu. Locret: Enhancing eviction in long-context LLM inference with trained retaining heads on consumer-grade devices. *arXiv preprint arXiv:2410.01805*, 2024. Accepted to TMLR.

[15] Hugging Face. Open R1: A fully open reproduction of DeepSeek-R1. https://github.com/huggingface/open-r1, 2025.

[16] Simon Jégou and Maximilian Jeblick. KVzap: Fast, adaptive, and faithful KV cache pruning. *arXiv preprint arXiv:2601.07891*, 2026.

[17] Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. KVzip: Query-agnostic KV cache compression with context reconstruction. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2505.23416.

[18] Jang-Hyun Kim, Dongyoon Han, and Sangdoo Yun. Fast KVzip: Efficient and accurate LLM inference with gated KV eviction. *arXiv preprint arXiv:2601.17668*, 2026.

[19] Adrian Łańcucki, Konrad Staniszewski, Piotr Nawrot, and Edoardo M. Ponti. Inference-time hyper-scaling with KV cache compression. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2506.05345.

[20] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024.

[21] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

[22] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning*, 2024.

[23] NVIDIA. kvpress: LLM KV cache compression made easy. https://github.com/NVIDIA/kvpress, 2024.

[24] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations*, 2024.

[25] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[26] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[27] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning*, 2024.

[28] Mengzhou Xia, Zexuan Zhong, and Danqi Chen. Structured pruning learns compact and accurate models. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2022.

[29] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.

[30] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. In *International Conference on Learning Representations*, 2025.

[31] Xinrong Zhang et al. ∞Bench: Extending long context evaluation beyond 100K tokens. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2024.

[32] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.

[33] Enshuai Zhou, Yifan Hao, Chao Wang, Rui Zhang, Di Huang, Jiaming Guo, Xing Hu, Zidong Du, Qi Guo, and Yunji Chen. LKV: End-to-end learning of head-wise budgets and token selection for LLM KV cache eviction. *arXiv preprint arXiv:2605.06676*, 2026.